

SUPPLEMENTARY PROBLEM: CLOSEST NEIGHBORS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem Statement

Assume you have a sorted array of integers A . You are given an integer k . k may or may not be in A . You want to output the pair of integers (a, b) where a is the largest integer in A that is smaller than k and b is the smallest integer in A that is larger than k . If no such number a or b exists, then the output should be $(null, b)$ or $(a, null)$, as appropriate. Note that there may be duplicated integers in the array. Develop an efficient algorithm to solve this problem.

Example:

Input: $A = [2, 4, 6, 8, 10]$, $k = 5$

Output: $(4, 6)$

Intuition

This question can be solved with one pass of binary search. However, for simplicity, this solution contains two passes of binary search (one for a and one for b). As you'll see, the asymptotic complexity is identical.

Pseudocode

Algorithm 1 Modified Binary Search

```
1: Input: List  $A$  of length  $n$ , target  $k$ 
2: Output: Pair  $(a, b)$  described in problem
3: procedure MODIFIED-BINARY-SEARCH( $A, k$ )
4:    $l, r \leftarrow 1, \text{length}(A)$ 
5:    $a \leftarrow \text{null}$ 
6:    $b \leftarrow \text{null}$ 
7:
8:   while  $l \leq r$  do ▷ First Loop (FL)
9:      $mid \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
10:    if  $A[mid] = k$  then
11:       $r \leftarrow mid - 1$ 
12:    else if  $A[mid] < k$  then
13:       $a \leftarrow A[mid]$ 
14:       $l \leftarrow mid + 1$ 
15:    else
16:       $r \leftarrow mid - 1$ 
17:    end if
18:  end while
19:
20:   $l, r \leftarrow 1, \text{length}(A)$ 
21:  while  $l \leq r$  do ▷ Second Loop (SL)
22:     $mid \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
23:    if  $A[mid] = k$  then
24:       $l \leftarrow mid + 1$ 
25:    else if  $A[mid] > k$  then
26:       $r \leftarrow mid - 1$ 
27:       $b \leftarrow A[mid]$ 
28:    else
29:       $l \leftarrow mid + 1$ 
30:    end if
31:  end while
32:  return  $(a, b)$ 
33: end procedure
```

Here is an example to visualize the pseudocode

	L R								k	a	b	
A	5	7	14	28	32	41	46	46	38	-	-	Start of FL

	L mid R								k	a	b	
A	5	7	14	28	32	41	46	46	38	-	-	FL Iteration 1 Place mid

	mid L R								k	a	b	
A	5	7	14	28	32	41	46	46	38	28	-	FL Iteration 1 $A[mid] < k$

	L mid R								k	a	b	
A	5	7	14	28	32	41	46	46	38	28	-	FL Iteration 2 Place mid

	R L mid								k	a	b	
A	5	7	14	28	32	41	46	46	38	28	-	FL Iteration 2 $A[mid] > k$

	R L								k	a	b	
A	5	7	14	28	32	41	46	46	38	28	-	FL Iteration 3 Place mid
					mid							

	R L								k	a	b	
A	5	7	14	28	32	41	46	46	38	32	-	FL Iteration 3 $A[mid] < k$
					mid							

	R L								k	a	b	
A	5	7	14	28	32	41	46	46	38	32	-	R < L FL Terminate
					mid							

	R				L				k	a	b	<i>Start of SL</i>
A	5	7	14	28	32	41	46	46	38	32	-	

	L			mid				R		k	a	b	<i>SL Iteration 1 Place mid</i>
A	5	7	14	28	32	41	46	46		38	32	-	

	mid				L			R		k	a	b	<i>SL Iteration 1 A[mid] < k</i>
A	5	7	14	28	32	41	46	46		38	32	-	

	L			mid				R		k	a	b	<i>SL Iteration 2 Place mid</i>
A	5	7	14	28	32	41	46	46		38	32	-	

	R				L			mid		k	a	b	<i>SL Iteration 2 A[mid] > k</i>
A	5	7	14	28	32	41	46	46		38	32	41	

	R				L			mid		k	a	b	<i>SL Iteration 3 Place mid</i>
A	5	7	14	28	32	41	46	46		38	32	41	

	R			L				mid		k	a	b	<i>SL Iteration 3 A[mid] < k</i>
A	5	7	14	28	32	41	46	46		38	32	41	

	R				L			mid		k	a	b	<i>R < L SL Terminate</i>
A	5	7	14	28	32	41	46	46		38	32	41	

Proof of Correctness

There are two loops in the pseudocode, therefore we have two loop invariants. These loops are similar though, so if the first L.I. can be found, the second one follows closely.

First Loop: Loop Invariant

At the start of each iteration,

1. $A[1..l-1] \leq k$
2. $A[r+1..n] \geq k$.
3. $a = \max\{A[1..l-1]\}$ s.t. $a < k$ (if it exists)

First Loop: Loop Invariant Proof

- **Initialization:** Assume $n = \text{length}(A)$. Before the loop, $l = 1$ and $r = n$ and $a = \text{null}$. Sub-arrays $A[1..0]$ and $A[n+1..n]$ don't exist. Therefore the statements in the L.I. are vacuously true.
- **Maintenance** Assume the L.I. is true for some l and r , s.t. $l \leq r$. There are three possible cases.

Case 1: $A[mid] = k$

Since l and a remain unchanged, Statements 1 and 3 of the loop invariant still hold. However, we update $r = mid - 1$. Given that $mid = r + 1$, and since $A[mid] = k$ in this case, it follows that $A[r+1..n] \geq k$ due to the non-decreasing order of the array. Therefore, Statement 2 holds, and so the entire loop invariant holds.

Case 2: $A[mid] < k$

Since r remains unchanged, the Statement 2 of the loop invariant still holds. However, we update $l = mid + 1$ and set $a = A[mid]$. Given that $mid = l - 1$, it follows that $A[1..l-1] \leq k$, since in this case $A[mid] < k$. So, Statement 1 still holds. For Statement 3, note that the array is sorted, so $A[mid] = A[l-1] = \max\{A[1..l-1]\}$. Since we set $a = A[mid]$ and $A[mid] < k$, the Statement 3 of the loop invariant also holds. Therefore, the loop invariant holds.

Case 3: $A[mid] > k$

Since l and a remain unchanged, Statements 1 and 3 still hold. However, we update $r = mid - 1$. Given that $mid = r + 1$, and since this case showed $A[mid] > k$, it follows that $A[r+1..n] \geq k$, and so Statement 2 still holds. Therefore, the loop invariant holds.

- **Termination** l starts from 1 and r starts from the length of the array, n . In each iteration, either l is increased or r is decreased, depending on the comparison between $A[mid]$ and k . Since the search interval $[l, r]$ shrinks monotonically and the array has finite length n , the

loop must eventually terminate.

At termination, $l > r$, which doesn't help us completely yet. First, consider two situations.

1. $l = r$ at beginning of the loop
2. $l = r - 1$ at beginning of the loop and $A[mid] < k$

In the first case, after calculating mid , trivially $mid = l = r$. Therefore, regardless of what branch is taken, at the beginning of the next iteration $l = r + 1$.

In the second case, after calculating mid , $mid = l = r - 1$. If $A[mid] < k$, our algorithm chooses to set $r = mid - 1$, which means at the next iteration $l = r + 1$.

So instead of claiming that $l > r$ at the end of the loop, we will claim that $l = r + 1$, as the situations given above are the only situations in which the loop terminates. From the L.I. we now know $a = \max\{A[1..r]\}$ s.t. $a < k$. We also know from the L.I. that $A[r + 1..n] \geq k$ which ensures that there are no elements greater than $A[r]$ and less than k . So at termination, we achieved the goal (which was to find the maximum element a s.t. $a < k$ and a is in the array).

Second Loop: Loop Invariant

At the start of each iteration,

1. $A[1..l - 1] \leq k$
2. $A[r + 1..n] \geq k$.
3. $b = \min\{A[r + 1..n]\}$ s.t. $b > k$ (if it exists)

Second Loop: Loop Invariant Proof

- **Initialization:** Assume $n = \text{length}(A)$. Before the loop, $l = 1$ and $r = n$ and $b = \text{null}$. Sub-arrays $A[1..0]$ and $A[n + 1..n]$ don't exist. Therefore the statements in the L.I. are vacuously true.
- **Maintenance:** Assume the L.I. is true for some l and r , s.t. $l \leq r$. There are three possible cases.

Case 1: $A[mid] = k$

Since r and b weren't changed at all, that means Statements 2 and 3 of the L.I. still hold. However, we set $l = mid + 1$. Since $mid = l - 1$, $A[1..l - 1] \leq k$ is true since this case showed $A[mid] = k$ and we have a non-decreasing sorted array. Therefore, Statement 1 is still true and the entire L.I. holds.

Case 2: $A[mid] > k$

Since l wasn't changed at all, that means Statement 1 still holds. However, we set $r = mid - 1$ and $a = A[mid]$. Since $mid = r + 1$, $A[r + 1..n] \geq k$ is true since this case showed $A[mid] > k$, so Statement 2 still holds. For Statement 3, note the array is sorted, so $A[mid] = A[r + 1] = \min\{A[r + 1..n]\}$. Since we set $a = A[mid]$ and $A[mid] > k$ in this case, Statement 3 holds.

Case 3: $A[mid] < k$

Since r and b weren't changed at all, that means Statements 2 and 3 still hold. However, we set $l = mid + 1$. Since $mid = l - 1$, $A[1..l - 1] \leq k$ is true since this case showed $A[mid] < k$, and Statement 1 still holds. Therefore, the L.I. holds.

- **Termination:** l starts from 1 and r starts from the length of the array, n . In each iteration, either l is increased or r is decreased, depending on the comparison between $A[mid]$ and k . Since the search interval $[l, r]$ shrinks monotonically and the array has finite length n , the loop must eventually terminate.

At termination, $l > r$, which doesn't help us completely yet. First, consider two situations.

1. $l = r$ at beginning of the loop
2. $l = r - 1$ at beginning of the loop and $A[mid] < k$

In the first case, after calculating mid , trivially $mid = l = r$. Therefore, regardless of what branch is taken, at the beginning of the next iteration $l = r + 1$.

In the second case, after calculating mid , $mid = l = r - 1$. If $A[mid] < k$, our algorithm chooses to set $r = mid - 1$, which means at the next iteration $l = r + 1$.

So instead of claiming that $l > r$ at the end of the loop, we will claim that $l = r + 1$, as the situations given above are the only situations in which the loop terminates. From the L.I. we now know $b = \min\{A[l..n]\}$ s.t. $b > k$. We also know from the L.I. that $A[1..l - 1] \leq k$ which ensures us there are no elements less than $A[l]$ and greater than k . So at termination, we achieved the goal which was to find the minimum element b s.t. $b > k$ and b is in the array.

Runtime Analysis

To prove the runtime of the algorithm, we can analyze the time complexity of each of the two loops and then combine our results to output a final runtime.

First While Loop (FL) — Finding the Largest a that is smaller than k

The first while loop is a binary search process, iterating to find the largest element a that is smaller than k .

At each iteration, the size of the search space is halved. The search space starts with the entire array of size n and reduces by half with each step, so the number of iterations is proportional to $\log n$. In each iteration, a comparison and assignment are performed in constant time $O(1)$. Thus, the time complexity of this first loop is $O(\log n)$.

Second While Loop (SL) — Finding the Smallest b that is larger than k

The second while loop is another binary search process. Like the first loop, the search space is halved at each iteration. The number of iterations is proportional to $\log n$, and each iteration performs constant time operations $O(1)$. Thus, the time complexity of this second loop is also $O(\log n)$.

Since both while loops execute in $O(\log n)$, and there are no nested loops, the total time complexity of the algorithm is the sum of the time complexities of the two loops:

$$O(\log n) + O(\log n) = O(\log n)$$

Thus, the overall runtime of the algorithm is:

$$O(\log n)$$